INFO 5 – Projet 1 : Analyse mathématique et statistique de textes

Niveau : L1 Maths | Séance 4 sur 4 (3h)

Objectifs de la séance

- Construire une fonction de répartition à partir d'une loi de probabilité discrète.
- Simuler une variable aléatoire par la méthode de la transformée inverse.
- Générer un texte aléatoire respectant une distribution statistique donnée.
- Rédiger un rapport de synthèse structurant les apprentissages en algorithmique.

Partie 1 : Probabilités cumulées (Fonction de répartition)

Pour générer aléatoirement une lettre en respectant sa probabilité d'apparition en français (par exemple, 14.7% de chances de tirer un 'e'), nous devons d'abord calculer les **fréquences cumulées**. Soit f le vecteur des fréquences. Le vecteur des fréquences cumulées C est défini par :

$$C_k = \sum_{i=0}^{k} f_i$$

Mathématiquement, la dernière case C_{25} doit valoir exactement 1.

Exercice 1 : Calcul des fréquences cumulées

```python
def probabilites_cumulees(frequences):
    """
    Transforme un vecteur de frequences (loi de probabilite)
    en un vecteur de frequences cumulees (fonction de repartition).
    """
    cumul = [0.0] * 26
    somme_en_cours = 0.0

    return cumul
```

Partie 2 : Simulation et Génération de texte aléatoire

En Python, la fonction `random.random()` du module `random` génère un nombre réel r uniformément réparti dans l'intervalle $[0, 1[$. Pour tirer une lettre selon notre loi de probabilité, il suffit de trouver le premier indice i tel que $C_i > r$.

Exercice 2 : Tirage d'une lettre (Méthode de la transformée inverse)

```python
import random

def tirer_lettre(cumul):
    """
    Tire une lettre au hasard en respectant la distribution de probabilites.
    """
    # Tirage d'un nombre aleatoire r dans [0, 1[
    r = random.________()
```

```
    # On cherche le premier indice ou le cumul depasse r


    # Par securite (aux erreurs d'arrondi pres), on retourne 'z' par defaut
    return 'z'
```

Exercice 3 : Le Singe Dactylographe

Le théorème du singe savant stipule qu'un singe tapant indéfiniment au hasard sur une machine à écrire finira par écrire les œuvres de Shakespeare. Écrivons une fonction pour générer un texte aléatoire de taille N.

```
def generer_texte(cumul, N):
    """
    Genere une chaine de caracteres de taille N.
    """
    texte_genere = ""


    return texte_genere

# Testez cette fonction en generant 500 caracteres avec les frequences du francais.
# Le resultat ressemble-t-il a des mots reels ? Pourquoi ?
```

Partie 3 : Rédaction du Rapport Final

En informatique, écrire un code fonctionnel n'est que la moitié du travail. Il est fondamental de savoir documenter son code, justifier ses choix d'implémentation et analyser l'efficacité de ses algorithmes.

Exercice 4 : Bilan et Synthèse Algorithmique

Vous devez rédiger un court rapport (format PDF, 2 à 3 pages maximum) résumant le travail de programmation réalisé durant ces 4 séances. Ce rapport ne doit pas être un simple copié-collé de votre code, mais une véritable analyse de votre démarche informatique.

Votre rapport devra obligatoirement suivre la structure suivante :

1. **Introduction :** Présentation globale du projet et des défis informatiques à relever (manipulation de chaînes de caractères, traitement de gros volumes de données, etc.).
2. **Structures de données et Implémentation :** Pour les grandes étapes du projet, expliquez **comment** vous avez codé les solutions :
   - **Choix des types :** Pourquoi et comment avez-vous utilisé des listes simples (vecteurs), des listes de listes (matrices 2D) ou des ensembles (`set`) selon les problèmes à résoudre ?
   - **Logique algorithmique :** Expliquez le fonctionnement de vos algorithmes principaux (par exemple, comment vos boucles imbriquées parcourent la matrice des bigrammes, ou comment fonctionne votre tri par insertion).
3. **Débogage et Difficultés techniques :**
   - Quels ont été les principaux obstacles rencontrés (erreurs d'indices `IndexError`, types incompatibles `TypeError`, problèmes d'indentation) ?
   - Comment avez-vous identifié et corrigé ces erreurs (utilisation de `print`, tests sur des petits mots, etc.) ?
4. **Analyse d'efficacité (Complexité) :** Quel algorithme vous a semblé le plus coûteux en temps de calcul ? Comment le temps d'exécution de votre programme évolue-t-il si l'on analyse un texte de 10 lignes versus un livre entier ?

5. **Conclusion :** Un bilan personnel sur vos progrès en programmation Python (ce que vous avez appris à faire, ce qui reste difficile pour vous).

Note : Le rapport sera évalué sur la clarté de vos explications informatiques, la pertinence de vos choix d'algorithmes, votre recul sur les erreurs rencontrées, et la qualité de la présentation.